

Отладка (Debug) и анализ производительности (профилирование)

Иванова Татьяна Владимировна
tvivanova@itmo.ru

Типы компиляции

- Debug – проект, пригодный для дебага (пошаговое выполнение, просмотр промежуточных переменных, и т.д.)
 - удобно отлаживать
 - медленно работает
- Release (выпуск) – проект без вспомогательной информации
 - работает быстро
 - пригоден для передачи заказчику
- ReleaseWithDebugInfo – проект работает так же быстро как релиз, но содержит некоторые вспомогательные данные
 - используется только для профилирования

Debug

- В режиме дебага можно посмотреть последовательность вызова функций и значения всех переменных

```

90 //-----
91 // сдвиг oSample_p для преобразования Фурье
92 void CalcFFT::ShiftSample(SampleComplex& oSample_p)
93 {
94     double pi = 2 * acos(0.);
95     for (int j = 0; j < oSample_p.GetSize(); j++)
96     {
97         for (int i = 0; i < oSample_p.GetSize(); i++)
98     {

```

Call Stack	
Name	
example_fft.exe!CalcFFT::ShiftSample(SampleComplex & oSample_p) Line 94	
example_fft.exe!CalcFFT::CalcFourier(SampleComplex & oSample_p) Line 73	
example_fft.exe!CalcFFT::Calc(const Param & oParam_p, Sample<double> & oFunction_p, Sample<double> & oR)	
example_fft.exe!QtWinCalcFourier::On_Calc() Line 101	
example_fft.exe!QtPrivate::FunctorCall<QtPrivate::IndexesList<>,QtPrivate::List<>,void,void (&cdecl QtWinCalcFourier::On_Calc() Line 101)>,void>::call<QtPrivate::List<>,void>(&cdecl QtWinCalcFourier::On_Calc() Line 101)>::operator()()	
example_fft.exe!QtPrivate::QCallableObject<void (&cdecl QtWinCalcFourier::On_Calc() Line 101)>,QtPrivate::List<>,void>::impl()	
[External Code]	
example_fft.exe!main(int argc, char * * argv) Line 10	
example_fft.exe!qtEntryPoint() Line 50	
example_fft.exe!WinMain(HINSTANCE__ * __formal, HINSTANCE__ * __formal, char * __formal, int __formal) Line 60	
[External Code]	

```

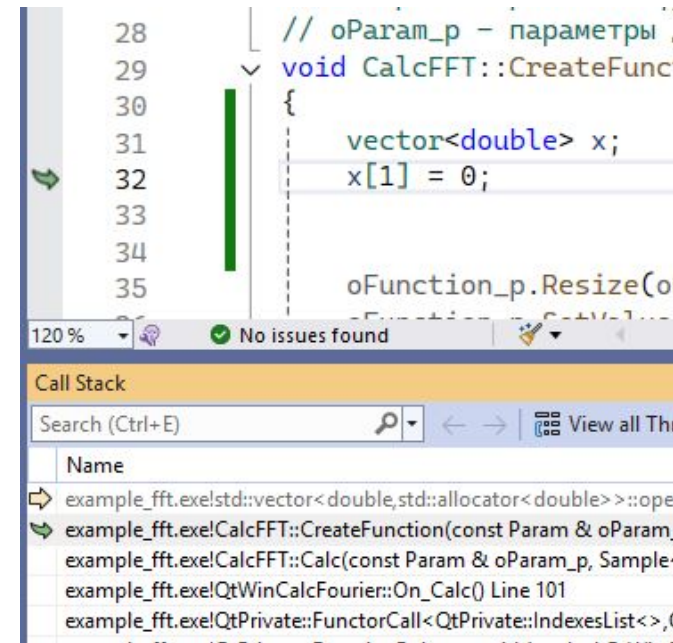
92 //-----
93 void QtWinCalcFourier::On_Calc()
94 {
95     // вычисления
96     CalcFFT Calc;
97
98     Get_Param_From_GUI();
99     Calc.Calc(m_Param, m_Func, m_Spectr);
100
101     on_m_PB_Redraw_clicked();
102 }

```

Autos		
Name	Value	Type
Calc	{...}	CalcFFT
m_Func	{m_size_x=0 m_size_y=0 m_data={ size=0 } }	Sample< double>
m_size_x	0	int
m_size_y	0	int
m_data	{ size=0 }	std::vector< double,std::allocator< double> >
m_Param	{m_dStepFunction=0.10000000000000001 m_dStepSpectr=0.0195312500...	Param
m_dStepFunction	0.10000000000000001	double
m_dStepSpectr	0.019531250000000000	double
m_iSize	512	int
m_dFunctionSize	1.0000000000000000	double
m_blsCircle	false	bool
m_Spectr	{m_size_x=0 m_size_y=0 m_data={ size=0 } }	Sample< double>
this	0x0000004b4215f7c0 {m_ui={...} m_Param={m_dStepFunction=0.100000...	QtWinCalcFourier *
QMainWindow	{...}	QMainWindow
m_ui	{...}	Ui::QtWinCalcFourier
m_Param	{m_dStepFunction=0.10000000000000001 m_dStepSpectr=0.0195312500...	Param
m_Func	{m_size_x=0 m_size_y=0 m_data={ size=0 } }	Sample< double>
m_Spectr	{m_size_x=0 m_size_y=0 m_data={ size=0 } }	Sample< double>

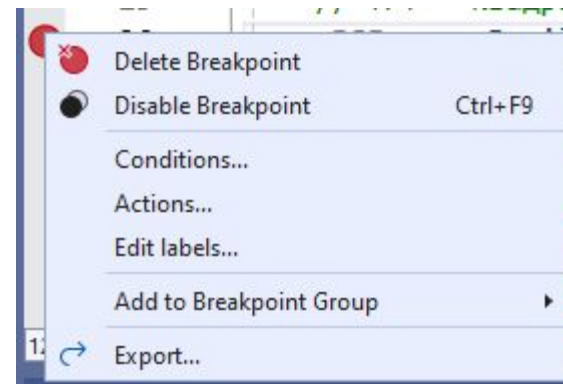
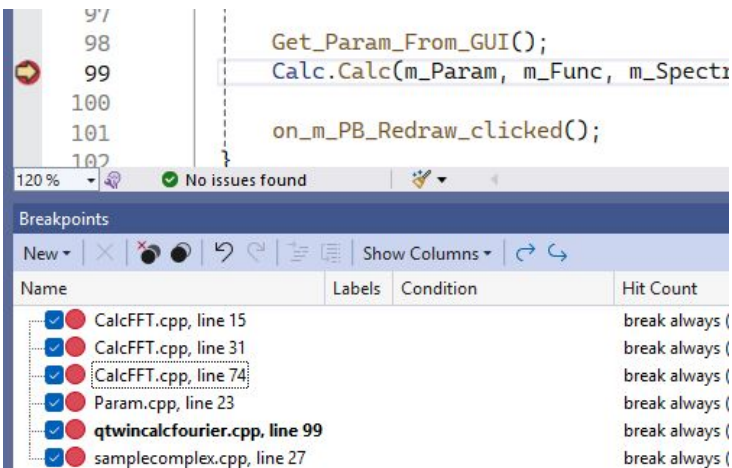
- ```
vector<double> x;
x[1] = 0;
```

- В Debug:
  - попадаем туда, где возникло исключение
  - смотрим последовательность вызовов
  - переходим куда надо (смотрим значения переменных, ставим точку прерывания)



# Инструменты Debug

- Пауза, окончание дебага 
- Выполнение по шагам
  - F11 зайти внутрь функции
  - F10 следующая строка кода
  - Shift+F11 - выйти из текущей функции
- Продолжить выполнение до следующей точки прерывания F5 
- Перейти к выбранной строке
- Управление точками прерывания
- Условие для точки прерывания
  - по правой кнопке мыши меню -> Conditions...-> задаем условие (например  $i == 99$ )



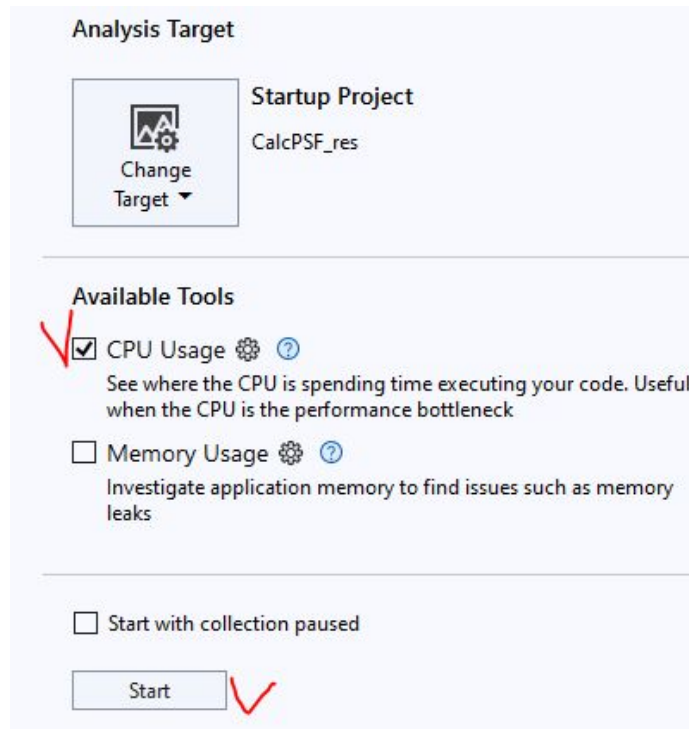
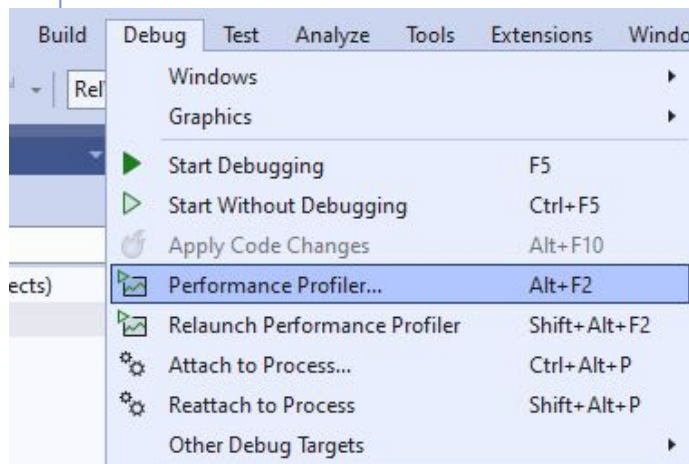
# Оптимизация производительности

- Выбор методов, алгоритмов, их параметров (шаг, кол-во итераций, и т.д.)
  - Оптимизация кода (профилирование)
  - Параллелизация
- 
- Профилирование имеет смысл когда:
    - программа работает без ошибок
    - программа работает слишком медленно
      - или может работать медленно при сочетании параметров
      - или хотим проверить все перед тем как отдать заказчику
    - лучше сразу писать оптимальный код



# Профилирование

- Выбираем RelWithDebInfo
- Строим проект
- Debug -> Performance Profiler



# Лабораторная работа №7

- Проанализировать скорость работы программы вычисления преобразования Фурье (л.р.№6) и выполнить оптимизацию кода по времени выполнения
  - выполнить профилирование в Visual Studio
  - найти неоптимальные куски кода и исправить их
    - минимум 3 изменения
    - если код уже хороший, и там нечего оптимизировать – можно внести неоптимальные куски
    - **цель работы – найти у себя неоптимальный код, а не ускорить программу любым способом!**
- В отчете привести:
  - время выполнения вычислений «до» и «после» оптимизации
  - исправленные куски кода программы
    - рядом, в два столбика «до» и «после» исправлений,
    - почему их надо было исправить, время
  - скрины этапов профилирования
  - файлы с итоговым кодом приложить к отчету